University of North Texas INFO4670.401 Fall 2025

Final Project

"Predicting student success in MOOCs: a comprehensive analysis using machine learning models"

Author: HosamA. Althibyani

Althibyani HA. 2024. Predicting student success in MOOCs: a comprehensive analysis using machine learning models. PeerJ Comput. Sci. 10:e2221
http://doi.org/10.7717/peerj-cs.2221

Sophie Krajmalnik
12-4-2025

Introduction

MOOCs are massive open online courses, an attractive and growing educational platform/format worldwide. These courses are available from within or without a connected university or academic institution. MOOCs are options for many students seeking alternative education or upskilling. These are easily accessible for anyone with an Internet connection. Students engaging in MOOCs offer a large amount of data, varying in their background, profile, and behaviors, that is helpful for furthering understanding of human behavior and educational systems. However, MOOCs often experience high dropout rates by students. While research has looked at modeling dropout prediction, little research has investigated measures of student success. This project looks to improve student retention and online education, promoting global education.

Research Question

This study leverages the vast amount of clickstream and assessment data generated on MOOC platforms to perform data mining and machine learning techniques on combined demographic, behavioral, and assessment data.

1. What attributes and characteristics contribute to student success?
2. Which form of model (logistic regression, random forest) can most accurately predict student outcomes (two or four classes)?

Methodology

Dataset Description

The data used in the study is sourced from the OULAD (Open University Learning Analytics) dataset, consisting of seven different tables. OULAD is from The Open University.

1. Courses – 22 entries with information on the course module, course presentation, and course length
2. Assessments – 206 entries with information on individual assessments and their type, by course module and presentation, as well as date of the assessment and its grading weight

3. Student Assessments – 173,912 entries with information on student performance on individual assessments
4. Student Information – 32,593 entries with demographic information on students
5. Student Registration – 32,593 entries with information on student registration by course module and presentation
6. VLE – 6,364 entries by site, course module, and presentation, showing types of activities
7. Student VLE – 1,0655,280 entries with information on student activities on VLE platforms by course module and presentation, showing number of clicks

Data Cleaning

After loading the tables and understanding the attributes of each, it was important to look at the schematic and see which attributes could be used like foreign and primary keys in a database.

Data was preprocessed and prepared for analysis in the following manner:

a. Merge Student VLE information with the VLE dataset using Student ID, Code Module, and Code Presentation to link the information properly.
b. Group the Merged VLE information by Student ID, Code Module, and Code Presentation, and within these unique groups, sum the total number of click interactions to determine VLE Features, and rename the column for clarity in the table.
c. Merge Student Assessment information with the Assessment dataset using ID Assessment to link the information properly.
    a. Convert attributes to the proper data type for future analysis—in this situation, the Score needed to be forced into a numeric type.
d. Calculate the Average Scores by grouping the Merged Assessments file by Student ID, Code Module, and Code Presentation, and finding the mean of the score for each unique group, then rename the column for clarity.
e. Merge the tables into one cohesive table for further analysis.
    a. Merge Student Information and Student Registration datasets using Student ID to link the information properly. This uses an inner join to return only the entries where there is a match in both tables by the keys provided.
    b. Merge Average Scores with the above table, using Student ID, Code Module, and Code Presentation to link the information properly. This uses a left join to return

all rows from the first merged table, along with matching rows from the Average Scores table by the keys provided.

   c. Finally, merge VLE Features with the above table, using Student ID, Code Module, and Code Presentation to link the information properly. This uses a left join to return all rows from the first merged table, along with matching rows from the VLE Features table by the keys provided.
   d. Convert attributes to the proper data type for future analysis—in this situation, the Date Registration and Date Unregistration needed to be forced into a numeric type.

f. Replace any null or missing values by the median value for that column attribute.
   a. Pull out the relevant numerical columns: Date Registration, Date Unregistration, Average Grade, Total Clicks, Number of Previous Attempts, Studied Credits.
   b. Use a SimpleImputer to perform the calculations and transformations onto all the numerical columns in the dataset.

g. Normalize the features within the range of 0 to 1. This helps prevent biased impacts from attributes that exist on a different scale.
   a. Use a MinMaxScaler to perform the calculations and transformations onto all the numerical columns in the dataset.

h. Use One Hot Encoding to expand the categorical attributes and make them numerically interpretable.
   a. Pull out the relevant categorical columns: Gender, Region, Code Module, Code Presentation, Highest Education, IMD Band, Age Range, Disability.
   b. Create a One Hot Encoder object that is prepared to handle sparse output, in order to convert to a usable DataFrame object.
   c. Pull out the categorical columns from the dataset into a new DataFrame.
   d. Use the One Hot Encoder to learn all unique categories for each column and transform the categorical columns into a binary indicator.
   e. Extract feature names in order to relabel columns for interpretability, and clean up the dataset.

Methods and Algorithms

Logistic regression was chosen as it is an effective model for predicting categorical outcomes and easily interpretable, though it is most useful for linear relationships. Random Forest was also chosen as it is useful for predicting multiple outcomes, can handle various types of variables, and can work with nonlinear relationships.

As described in the paper, a 75/25 train/test split was used, and both a 4-class and minimized 2-class predictions were run for both the logistic regression and random forest model. Mapping functions were used for this conversion, not only to combine results but to convert to numerical features. Two different datasets resulted from these efforts.

Model Training and Validation

Using the 75/25 split, testing and training sets for both the independent and dependent variables were created for each class of prediction. The dependent variable was the result, while the independent variables were the datasets without the result. This allowed the model to learn what contributes to student success or failure properly. Models were then fitted on the training data and applied to the testing data it had never seen before to determine accuracy of the model. Model performance was tested with accuracy measures, F1 score, and confusion matrix. The F1 score combines the rates of false positives and false negatives seen in a confusion matrix.

Results and Discussion

The four class logistic regression model returned an accuracy of 37.7%, which is remarkably low, and an F1 score of 0.207. The confusion matrix shows that it failed to identify a good portion of the classes, likely due to the issues with the logistic regression failing to converge.

The two class logistic regression model performed better, seeing an accuracy of 52.8% and an F1 score of 0.364, but this accuracy is still only slightly better than chance. The confusion matrix, however, reveals that the model collapsed and could only predict the majority class (due to the two class combination, the data was biased).

The four class Random Forest model achieved a much better accuracy of 80.9% with an F1 score of 0.803. The confusion matrix is more balanced, but still has boxes with zeroes,

suggesting that the model is missing information necessary to accurately predict an entire category in one way or another.

The two class Random Forest model performs overall the best, with an accuracy of 88.9% and an F1 score of 0.889, as well as a balanced confusion matrix.

As expected, the Random Forest models performed better, as they can handle the nonlinear relationships better, and did not have the student-specific issue of convergence failure. However, none of the models performed comparably to the paper this was to model, due to the errors discussed below. Random Forest models are also useful as we can move forward to extract the key features and contributors, which can assist in the goal of determining what predicts student success and improving overall education.

Conclusions and Limitations

This analysis does not compare to the paper it is supposed to emulate, based on the following issues:

1. **Logistic Regression failed to converge**. In order to even get the model to run and predict, I had to use such a small number of iterations that it never converged and has extremely low accuracy. When attempting to use a higher number of iterations, the model never fully executed. This could be helped by incorporating the steps in the next few points as well as performing a dimensionality reduction technique.
2. **Missing Features described in paper.** I was incredibly confused by the features we were supposed to develop from the datasets. Gemini helped me determine that I was missing VLE Activity Type Features, which it stated were the most predictive features.

AI Collaboration Reflection

Verification

AI was wrong quite a bit, to the point of frustration. I had to provide my code, specific actions I had taken, and features in the dataset in order to provide all the necessary context to begin to get proper information. It also frequently misunderstood what I was suggesting, such as when I had to explain that my merged dataset kept missing the activity type inclusion.

Partner Moment

Some of the most helpful moments were explaining what certain Python functions could and couldn't do, generating artifacts like checklists so I could verify my progress, explaining error messages, and explaining vocabulary and concepts in the context of this paper.

This question was essential to my work with the analysis and with AI: "I don't understand the point of the 4 vs 2 class classification. Can you explain its purpose and how to implement?"

Gemini provided me an answer both for the paper itself and for general thinking I can apply to other projects: "...The core purpose of the distinction is to balance between detailed prediction and high predictive accuracy for critical decisions…4-Class Classification (Detailed Prediction)...To predict the most granular outcome for a student. This model aims for maximum detail, helping instructors understand if a student is likely to withdraw early, simply fail, or achieve an excellent grade (distinction)…2-Class Classification (Actionable Prediction)...To simplify the problem to the most critical, actionable decision: Is the student on a path to success or failure?"

Crutch Check

There were various moments in which I wanted AI to be able to generate all my code for me, so it could use its own logic. However, I wanted to be able to understand what I was doing, even if it wasn't the most efficient manner. I wanted to stick with methods and code I could understand, which meant I was missing a few features in my pipeline and modeling.

References

Althibyani HA. 2024. Predicting student success in MOOCs: a comprehensive analysis using machine learning models. PeerJ Comput. Sci. 10:e2221 http://doi.org/10.7717/peerj-cs.2221